Jajapy: a learning library for stochastic models (Extended) *

Raphaël Reynouard Anna Ingólfsdóttir Giovanni Bacci

July 10, 2023

Abstract

We present Jajapy, a Python library that implements a number of methods to aid the modelling process of Markov models from a set of partially-observable executions of the system. Currently, Jajapy supports different types of Markov models such as discrete and continuous-time Markov chains, as well as Markov decision processes.

Jajapy can be used both to learn the model from scratch or to estimate parameter values of a given model so that it fits the observed data the best. To this end, the tool offers different learning techniques, either based on expectation-maximization or state-merging methods, each adapted to different types of Markov models. One key feature of Jajapy consists in its compatibility with the model checkers Storm and Prism.

The paper briefly presents Jajapy's functionalities and reports an empirical evaluation of their performance and accuracy. We conclude with an experimental comparison of Jajapy against AALpy, which is the current state-of-the-art Python library for learning automata. Jajapy and AALpy complement each other, and the choice of the library should be determined by the specific context in which it will be used.

1 Introduction

Markov models are a very popular formalism. Discrete-time Markov chains (MCs) and continuous-time Markov chains (CTMCs) have wide applications in performance and dependability analysis, whereas Markov decision processes (MDPs) are key models for stochastic decision-making and planning which find numerous applications in the design and analysis of cyber-physical systems.

Prism [1] and Storm [2] are two widely-used model checking tools that provide an efficient and reliable way to verify the correctness of probabilistic systems. They both accept models written in the Prism language, an expressive state-based language based on [3]. Prism is a powerful tool for modeling

*Anna Ingólfsdóttir and Raphaël Reynouard are supported by the *Learning and Applying Probabilistic Systems* project (Project nr. 206574-051) funded by the Icelandic Research Fund. This paper is an extended version of a paper accepted at QEST'23

and analysing MCs, MDPs, and probabilistic timed automata. It has a user-friendly interface and supports a variety of analysis techniques, including model checking, parameter synthesis, and probabilistic model checking. Storm, on the other hand, is a highly scalable and efficient tool for analysing probabilistic systems with continuous-time and hybrid dynamics [4]. It supports both explicit and symbolic model representation, and provides state-of-the-art algorithms for model checking and synthesis tasks. Both tools have been extensively used in academia and industry to analyse a wide range of systems, including communication protocols, cyber-physical systems, and biological systems.

The standard assumption of model checking tools is that the model is known precisely. For many application domains, this assumption is too strong. Often the model is not available, or at best is partially known. In such cases, the model is typically estimated empirically from a set of partially-observable executions (a.k.a. traces). Depending on the system under consideration, traces may be collected offline in the form of time series or (possibly continuous) streams of system logs, or the modeller can actively query the system and stir the exploration of its dynamics. In the latter situation, interaction with the system may be limited due to safety critical concerns, or simply to comply with the budget allocated for the task.

To effectively exploit the characteristics of different learning scenarios it is convenient to have a single library that provides a variety of learning algorithms, which can handle different learning scenarios and model types seamlessly, while integrating well with the model-and-verification workflow of Prism and Storm.

In this paper, we present Jajapy [5], a free open-source Python library that offers a number of techniques to learn Markov models from traces and is interoperable with Prism and Storm. Jajapy implements the following machine-learning techniques:

(i) Alergia [6, 7] and IOAlergia [8, 9], passive learning procedures that learn respectively MCs and (deterministic) MDPs from a set of traces by successively merging compatible states;

(ii) a number of adaptations of the Baum-Welch algorithm [10] to learn MCs, MDPs [11, 12], and CTMCs [13] by estimating their transition probabilities given a set of traces and the size of the resulting model;

(iii) active learning strategies to enhance the quality of the MDPs learned using the Baum-Welch algorithm [11, 12] when the user has the possibility to interact with the system;

(iv) MM algorithms [13] for estimating parameter value in parametric CTMCs (pCTMCs) from a set of (possibly non-timed) traces.

Jajapy implements also metrics to independently evaluate the output model against a test set. This is particularly useful to measure the degree of generalisation that the output model offers on top of the training set and assess whether the output model overfits the training data or not. Interoperability with Prism

and Storm is achieved by supporting import and export functions for Prism models as well as Stormpy sparse models.

Jajapy's source code follows a modular architecture design and can therefore be extended to other modelling formalisms and learning algorithms. Jajapy's documentation can be found on Read the Docs [14] that is complemented with a short video-introduction available on Zenodo [15].

Related work. AALpy [16] is a recent Python library that can learn both non-stochastic and stochastic models. In particular, AALpy can learn MDPs using L^*_{MDP} [17], an extension of Angluin's L^* algorithm [18], and MCs using Alergia [6, 7]. In Section 5.3, we compare Jajapy and AALpy performance.
Other automata learning frameworks have been developed as well. For example, Learnlib [19] and libalf [20], which learn non-stochastic models. In contrast with these tools, as of now, Jajapy primary focus is on learning Markov models.
In this paper, we present the different sampling methods implemented in Jajapy and AALpy. However, other methods also exist, such as the MDI algorithm [21] and [22], two state-merging based approaches, or the Bayesian method using Gibbs sampling [23] proposed by Neal in [24].
MDPs are extensively used in *reinforcement learning* as in [25, 26, 27], and in *robust reinforcement learning* [28] as in [29, 30, 31]. In this context, the objective is to learn an optimal policy that maximises long-term rewards in a given environment.
A related line of research is model synthesis. Counterexample-guided inductive synthesis (CEGIS) [32] study the problem of completing a given program sketch (i.e., a probabilistic program with holes) so that it satisfies a given set of quantitative specifications. Another approach is parameter synthesis [33, 34], where the objective is to find some (or all) instances of a given parametric Markov model satisfying a logic formula. In [35], the authors combine parameter synthesis and parametric inference techniques to synthesize feasible parameter valuations and quantify the confidence that the corresponding model satisfies a given property of interest.

Paper outline. We start with a quick introduction to Jajapy functionalities in Section 2, then we explain Jajapy's features from a theoretical perspective in Section 3. In Section 4, we present some technical aspects of Jajapy, and in Section 5 we evaluate our tool and compare it to AALpy.

2 Jajapy in a nutshell

Jajapy offers learning methods to construct an accurate model of a system under learning (SUL) from a set of traces and export it to a format that can be directly used in Storm and Prism for analysis.

In the following, we call *training set* (resp. *test set*) the collection of traces used to learn the SUL model (resp. to evaluate the learning output model). Depending on the nature of the training set, Jajapy learns different types of

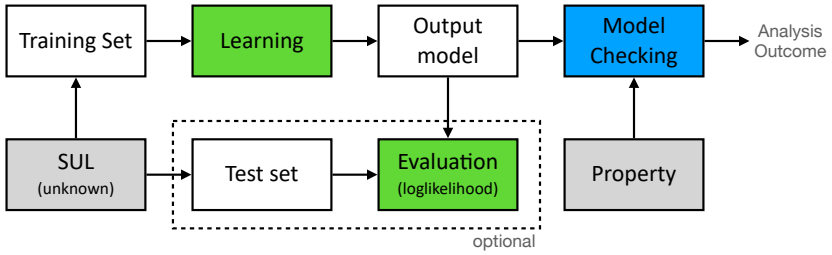

Figure 1: A complete modeling and verification workflow using JAJAPY . The phases where JAJAPY is employed are highlighted in green, whereas the phase in blue is assumed to be performed with STORM or PRISM.

```
from jajapy import BW
type(training_set) # list
output_model = BW().fit(training_set, nb_states=10)
type(output_model) # stormpy.SparseDtmc
```

Figure 2: Simple execution of JAJAPY BW to learn an MC with 10 states.

models: (i) MCs are learned from sequences of labels (i.e., sequences of atomic propositions), (ii) CTMCs and pCTMCs are learned from times series of labels, and (iii) MDPs are learned from alternating sequences of actions and labels. A trace denotes, depending on the context, a sequence of labels, a time series of labels, and an alternating sequence of actions and labels. The length of a trace is always the number of labels it contains.

The first and main learning algorithm offered by JAJAPY is the Baum-Welch (BW) algorithm. It takes as input a training set and an initial hypothesis, i.e. a Markov model. During the BW execution, the transition probabilities of this initial hypothesis will be updated but no state will be added/removed from it. Therefore, the number of states in the resulting model will be equal to the number of states in the initial hypothesis. By default JAJAPY generates a random initial hypothesis (given as input the number of states) but the user can also provide one explicitly. This enables the user to exploit his knowledge of the SUL to enhance the learning process. Such an initial hypothesis can be a STORMPY sparse model, a model saved in a PRISM file or a JAJAPY model. Given a training set and an initial hypothesis, the BW algorithm constructs an approximate representation of the SUL, called *output model*.

As an alternative to the BW algorithm, JAJAPY offers implementations of Alergia and IOAlergia to learn respectively MCs and MDPs. These algorithms take as input the training set and a *confidence* parameter.

Once JAJAPY has produced the output model, the user can use STORMPY to verify the model against some properties of interest supported by STORM. The output model can also be exported to a PRISM model and analysed with the PRISM model checker.

3 Learning probabilistic models

In this section, we briefly describe the key characteristics of the learning methods for Markov models currently available in JAJAPY and AALPY.
These methods belong to two categories, active and passive. *Active* learning methods learn from interactions with the SUL, while *passive* methods learn from the training set only. Active learning methods are usually more efficient (in terms of data), but can be used only if it is possible to interact with the SUL.

Some learning methods allow the user to decide the size (i.e. the number of states) of the output model, preventing the algorithm from generating models too large to be efficiently analysed. The downside of such a feature consists in the fact that, if the number of states requested is too large (resp. small), the output model may overfit (resp. underfit) the training set.
A Markov model is *deterministic* if, for any state s and label ℓ, there exists at most one transition leaving s to a state labelled with ℓ. Some of the learning methods described below assume the SUL to be deterministic. When such methods are exercised with a SUL that is non-deterministic, they are not guaranteed to converge to the true model, instead, they will return a deterministic model that approximates the SUL. Typically, the approximated model is larger than the SUL.

Expectation Maximisation approach. The Baum-Welch (BW) algorithm is an iterative maximum likelihood estimation method to estimate the parameters of Markov models [36]. This technique is an application of the Expectation Maximisation algorithm. Originally designed for Hidden Markov Models [10], it has been adapted to MCs, CTMCs and MDPs [12, 13].
Given a set of traces $\mathcal{O}$ (the training set) and an initial hypothesis $\mathcal{H}_0$, the BW algorithm iteratively updates $\mathcal{H}_0$ such that the likelihood that the hypothesis generates $\mathcal{O}$ has increased with respect to the previous step. The algorithm stops when the likelihood difference between two successive hypotheses is lower than a fixed threshold ϵ. In JAJAPY, the user can also set an upper bound on the number of BW iterations. BW converges to a local optimum [37].
The BW algorithm is a passive learning approach, it allows the user to decide the size of the output model, and can learn non-deterministic models.

Active learning with sampling strategy. JAJAPY implements an active learning extension of the BW algorithm for MDPs [11, 12]. This method uses a sampling strategy to generate new training samples that are most informative for the current model hypothesis. With this method, the user decides the size of the output model. This algorithm is able to learn non-deterministic models. Currently, JAJAPY only supports the sampling strategy described in [11, 12].

State-merging approach. Both JAJAPY and AALPY provide an implementation of the Alergia algorithm [6, 7] to learn MCs and its extension IOAlergia

[9] to learn MDPs. These algorithms use a state-merging approach. Starting from a maximal tree-shaped probabilistic automaton representing the training set, they iteratively merge states that are "similar enough" according to an Hoeffding test [38]. The accuracy of the Hoeffding test is provided as input. These algorithms are passive, they do not allow the user to choose the number of states in the output model, and they assume the SUL to be deterministic.

Active learning with membership and equivalence queries. AALpy provides an implementation of L^*_{MDP} [17], an extension of Angluin's L^* algorithm [18] to learn MDPs. As for Alergia, this method assumes the SUL to be deterministic, and the size of the output model cannot be chosen in advance.

Table 1 summarises the key characteristics of the learning methods discussed above. The 5th column indicates whether or not the user can choose the number of states in the output model, and the 6th column indicates whether the algorithm is able to generate non-deterministic models or not.

Algorithm	**Model**	**Reference**	**Active**	**# states**	**Non-det.**	Jajapy	AALpy
BW-MC	MC	[11, 12]	✗	✓	✓	✓	✗
BW-CTMC	CTMC	[13]	✗	✓	✓	✓	✗
MM-pCTMC	pCTMC	[13]	✗	✓	✓	✓	✗
BW-MDP	MDP	[11, 12]	✗	✓	✓	✓	✗
Active-BW	MDP	[11, 12]	✓	✓	✓	✓	✗
Alergia	MC	[6, 7]	✗	✗	✗	✓	✓
IOAlergia	MDP	[8, 9]	✗	✗	✗	✓	✓
L^*_{MDP}	MDP	[17]	✓	✗	✗	✗	✓

Table 1: Key characteristics of the selected learning algorithms for Markov models.

4 Architecture and technical aspects

In this section, we describe some internal aspects of Jajapy.

Jajapy models. In Jajapy, each kind of model is represented by a class, which inherits from an abstract class `Model`. This makes Jajapy modular and easy to extend to other model formalisms.
The abstract class `Model` implements the methods to run the model, generate traces and compute the loglikelihood [1] of a set of traces under the model. Currently, all models in Jajapy use an explicit state-space representation.
Every Jajapy model has an attribute `matrix` which contains the transition

[1] the logarithm of the likelihood

probabilities. This `matrix` is a *Numpy ndarray* [39] of floats. Each JAJAPY model has also an attribute `labelling` containing the label associated to each state. This attribute is a Python list whose length equals to the number of states of the model. Finally, JAJAPY uses *Sympy* [40] to represent symbolic expressions used for transition rate expressions in pCTMCs.

Learning with BW. BW executions are handled by the `BW` class.
The `BW.fit` method starts by determining which model formalism should be used according to the given initial model (if provided) and the training set.
Then, it selects the appropriate update procedure and runs the BW algorithm.
An execution of the BW algorithm resolves into a sequence of matrix operations that are handled by Numpy. In addition, if JAJAPY is executed on a Linux machine, it supports multithreading to speed up the BW algorithm: at each BW iteration, JAJAPY executes one thread for each unique trace in the training set.

Output models. The output format of any JAJAPY learning methods can be chosen among the following: STORMPY sparse model or JAJAPY model. The output model can also be exported to a PRISM file by setting the `output_file_prism` parameter of the `BW.fit` method.

Representing training sets and test sets. JAJAPY uses its own `Set` class to represent training and test sets. This class has two attributes: (i) `sequences`, the set of all unique traces in the training set, and (ii) `times`, which contains, for each trace in `sequences`, the number of times this trace has been observed. This reduces significantly the number of computations during the learning process when traces appear several times in the training set. Nevertheless, the training set can be given as a Python list (or *Numpy ndarray*) to the `BW.fit` method.
In Jajapy, training sets and test sets are not represented through a prefix tree (as is normal in other libraries) since this is only advantageous when Jajapy is used in single thread mode. The training sets/test sets are sorted by Jajapy only in this case.

5 Experimental evaluation and comparison

In this section, we first test JAJAPY validity. Secondly, we empirically evaluate how the different learning methods scale with the size of the output model and the training set. Finally, we compare it with AALPY.

All the experiments were run on a Linux machine with an AMD Ryzen 9 3900X 12-Core processor and 32 GB of memory.

In the experiments, we use the *loglikelihood distance* (abbreviated as *ll. distance*) as a metric to compare two models: given two models $\mathcal{M}$ and $\mathcal{M}'$ and n traces $\mathcal{O}$, the loglikelihood distance w.r.t. $\mathcal{O}$ is $\frac{1}{n}|\ln L(\mathcal{M},\mathcal{O}) - \ln L(\mathcal{M}',\mathcal{O})|$, where $L(\mathcal{M},\mathcal{O})$ denotes the likelihood of $\mathcal{O}$ under $\mathcal{M}$.

5.1 Jajapy validation testing

We test Jajapy validity as follows: (i) we translate a Stormpy model $\mathcal{M}$ representing the Yao-Knuth's die [41] to a Jajapy one; (ii) we use it to generate a training set of 10,000 traces of length 10: 10 being big enough to reach the final state with a decent probability and 10,000 being small enough to learn the model in few seconds, but sufficiently big to learn a correct approximation of the SUL; (iii) we learn, using Jajapy BW and Alergia implementations, two new Stormpy models $\mathcal{M}'$ and $\mathcal{M}''$, and finally (iv) $\mathcal{M}', \mathcal{M}''$ are compared both w.r.t. their outcomes on some relevant model checking queries and their loglikelihood distance on a test set relative to the true model $\mathcal{M}$.

The first three queries correspond to the probability that the die roll gives us 1, 2 or 3. The next three queries indicate the probability that the die gives us 4, 5 or 6 without ever going through the same state (except the final one) more than once. Finally the last query corresponds to the probability that 10 throws of the coin are enough to simulate the roll of the die.

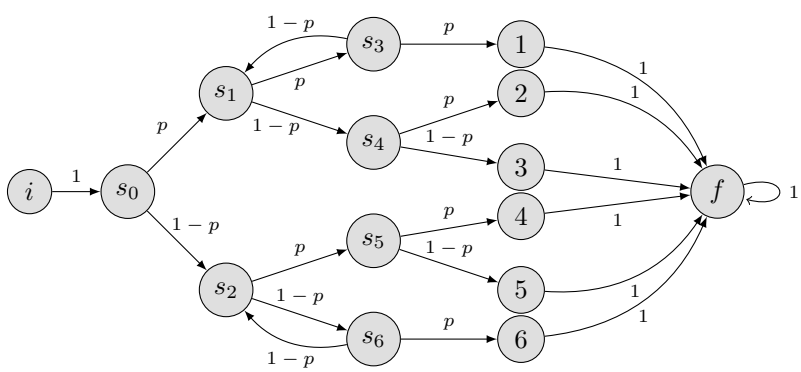

Figure 3: The Yao-Knuth's die from [41]

We run these experiments on a Markov chain modelling the Yao-Knuth's die represented in Figure 3 once with $p = 0.5$ (i.e. with a unbiased coin) and once with $p = 0.9$. Table 2 and 3 show that Jajapy learned a valid representation of the source model regardless of the algorithm used. When the coin is unbiased, Alergia learns a bigger model than BW (23 states against 14) which is better in terms of loglikelihood distance but worst for the model checking queries. This is explained by the fact that, in this case, Alergia is not able to merge a large number of states and, therefore, generates a model close to a PTA, which is efficient in terms of ll distance (especially when the sequences in the test set are the same length as those in the training set). When the coin is biased, some possible traces do not appear or appear very little in the training set. Therefore, the training set being composed of much more similar traces, the initial PTA is much smaller as well as the model generated by Alergia. On the other hand, the likelihood of the sequences present in the test set and not in the training set can be fairly different between the model generated by Alergia and the SUL. In

	true	BW	Alergia
# states	14	**14**	23
$\mathbb{P}(F(1))$	0.167	0.168	0.168
$\mathbb{P}(F(2))$	0.167	0.170	0.169
$\mathbb{P}(F(3))$	0.167	0.163	0.163
$\mathbb{P}(F^{\leq 4}(4))$	0.125	**0.130**	0.143
$\mathbb{P}(F^{\leq 4}(5))$	0.125	**0.124**	0.136
$\mathbb{P}(F^{\leq 4}(6))$	0.125	0.107	**0.129**
$\mathbb{P}(F^{\leq 10}(f))$	0.996	0.979	**0.973**
ll. distance	0.0	1.700	**1.616**
learning time (s)	-	1.039	**0.003**

Table 2: Results for an unbiased Yao-Knuth's die ($p = 0.5$).

	true	BW	Alergia
# states	14	**14**	12
$\mathbb{P}(F(1))$	0.801	0.797	0.797
$\mathbb{P}(F(2))$	0.089	0.092	0.092
$\mathbb{P}(F(3))$	0.010	0.008	0.008
$\mathbb{P}(F^{\leq 4}(4))$	0.081	0.088	**0.076**
$\mathbb{P}(F^{\leq 4}(5))$	0.009	0.010	**0.009**
$\mathbb{P}(F^{\leq 4}(6))$	0.001	0.002	0.002
$\mathbb{P}(F^{\leq 10}(f))$	1.0	**0.999**	0.992
ll. distance	0.0	**0.511**	1.569
learning time (s)	-	1.060	**0.001**

Table 3: Results for a biased Yao-Knuth's die ($p = 0.9$).

other words, for the same training set, a model generated by Alergia will often be less general than one generated by BW, because Alergia is more sensitive to overfitting.

5.2 Experimental evaluation of the scalability

5.2.1 Scalability evaluation for MCs, CTMCs and MDPs.

To evaluate the scalability of our software, we report the running time and the memory footprint required to learn models with an increasing number of states.

We use Jajapy to learn randomly generated transition-labeled MCs and CTMCs ranging from 10 to 200 states, corresponding to models with up to 100 to 40,000 parameters (the number of parameters is at most s^2, where s is the number of states). We perform the same experiment for MDPs with 5 to 100 states and 4 actions, thus having at most 100 to 40,000 parameters (here the number of parameters is at most $s^2 \cdot a$, where s and a are respectively the number of states and actions). We employ training sets containing 1,000 traces of length 10. These two values offer a good compromise between accuracy and running time. We set the size of the initial hypothesis equal to that of the SUL. The results are shown in Fig. 4.

The running time for all type of SULs increases exponentially, but at a larger rate for CTMCs: while one BW iteration for an MDP with 200 states and an MC with 400 states took around two minutes in this setting, one BW iteration for a CTMC with 200 states took 97 minutes. This is due to the computational difficulty of calculating rates of exponential distributions when learning CTMCs parameters. Memory usage also increases exponentially for all types of Markov models. These exponential growths were expected, since the number of parameters to estimate increases exponentially with the number of

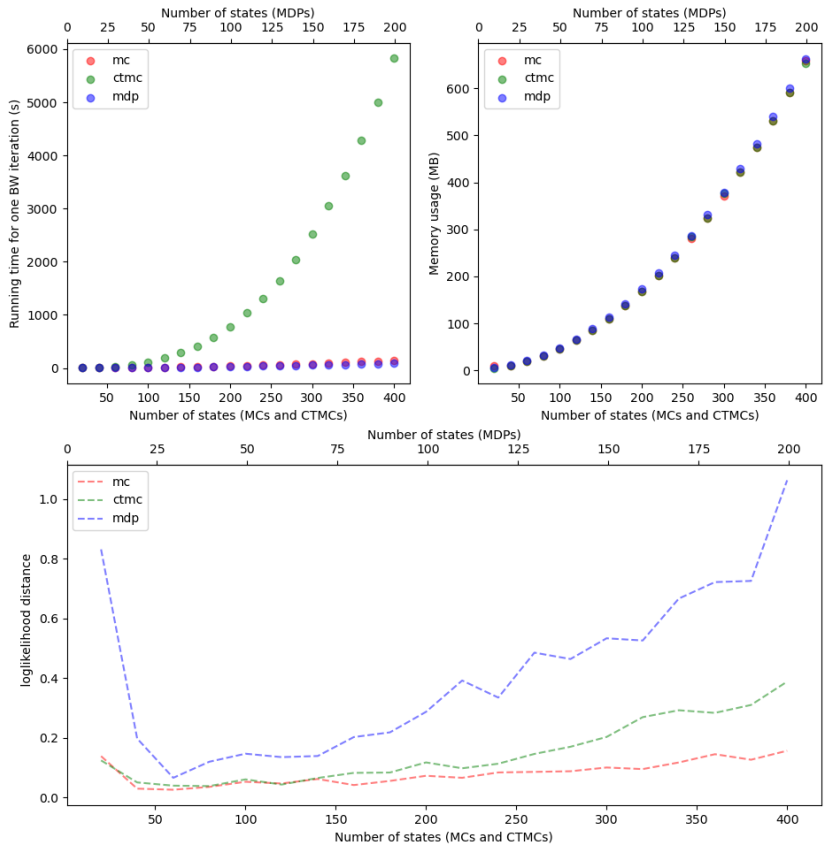

Figure 4: JAJAPY running time, memory usage and loglikelihood distance w.r.t. the number of parameters of the hypothesis.

```
ctmc
 // Tandem Queuing Network [Hermanns, Meyer-Kayser & Siegle]
const int c;  // queue capacity
const double lambda = 4 * c;
 // model parameters
const double mu1a = 0.2; const double mu1b = 1.8; const double mu2 = 2; const double kappa = 4;

module serverC
    sc : [0..c] init 0;
    ph : [1..2] init 1;

    [] (sc<c) → lambda : (sc′=sc + 1);
    [route] (sc>0) & (ph=1) → mu1b : (sc′=sc − 1);
    [] (sc>0) & (ph=1) → mu1a : (ph′=2);
    [route] (sc>0) & (ph=2) → mu2 : (ph′=1) & (sc′=sc − 1);
endmodule

module serverM
    sm : [0..c] init 0;

    [route]    (sm<c) → 1 : (sm′=sm + 1);
    [] (sm>0) → kappa : (sm′=sm − 1);
endmodule
```

Figure 5: Prism model for the tandem queueing network from [42].

states.
Finally, as the complexity of the model increases, the loglikelihood distance grows. This issue is usually mitigated by increasing the length and number of traces in the training set.

5.2.2 Scalability evaluation for pCTMCs.

To evaluate the scalability of our software on pCTMCs, we use the tandem queueing network model from [42] (*cf.* Fig. 5) as a benchmark for our evaluation.

The experiments have been designed according to the following setup. We assume that the state of `serverC` is fully observable —i.e., its state variables `sc` and `ph` are– as well as the size `c` of the queue and the value of `lambda`. In contrast, we assume that the state of `serverM` is not observable.

Each experiment consists in estimating the value of the parameters `mu1a`, `mu1b`, `mu2`, and `kappa` from a training set consisting of 100 traces of length 30, generated by simulating the Prism model depicted in Fig. 5. When the value of `c` is large, it is necessary to have lengthy traces to cover the state space of the SUL. As a result, traces with a length of 30 are utilised. However, in order to restrict the amount of time taken for execution, only 100 traces are employed. We perform this experiment both using timed and non-timed observations, by increasing the size `c` of the queue until the running time of the estimation exceeds a time-out set to 1 hour. We repeat each experiment 10 times by randomly re-sampling the initial values of each unknown parameter x_i in the interval [0.1, 5.0]. We annotate the running time as well as the relative error δ_i for each parameter x_i, calculated according to the formula $\delta_i = |e_i - r_i|/|r_i|$, where e_i and r_i are respectively the estimated value and the real value of x_i.

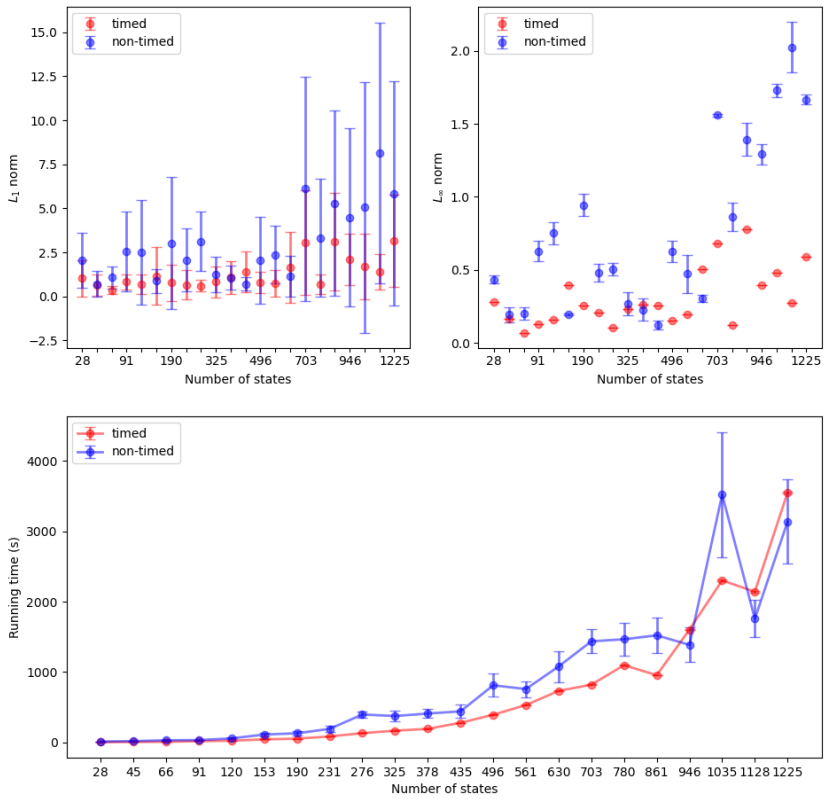

Figure 6: Comparison of the performance of the estimation for timed and non-timed observations on the tandem queueing network with different size of the queue.

Fig. 6 (bottom) depicts the graph of the average running time relative to the model size together with error bars. We observe that the running time is quadratic in the number of states (equivalently, linear in the size the number of states plus the number of non-zero transitions of the model) both for timed and non-timed observations. However, for non-timed observations, the variance of the measured running times tends to grow with the size of the model. In this respect, we observed that large models required more iterations than small models to converge. Nevertheless, all experiments required at most 20 iterations.

Fig. 6 (top) details the average L_1-norm (resp. L_∞-norm) of the vector $\delta = (\delta_i)$, calculated as $\|\delta\|_1 = \sum_i |\delta_i|$ (resp. $\|\delta\|_\infty = \max_i |\delta_i|$). As one may expect, the variance of the measured relative errors is larger in the experiments performed with non-timed observations, and the quality of the estimation is better when employing timed observations. Notably, for timed observations, the quality of the estimation remained stable despite the size of the model increased relative to the size of the training set. This may be explained by the

fact that the parameters occur in many transitions.

5.3 Comparison with AALpy

AALpy is an active automata learning python library. AALpy implements several learning algorithms to learn various families of automata. Since Jajapy learns stochastic models only, we will focus here on these models. However, AALpy is also able to learn non-stochastic models, as Deterministic Finite Automata or Mealy Machines.
AALpy implements L^*_{MDP}, an extension of Angluin's L^* algorithm [17, 18] to learn MDPs, and the Alergia algorithm to learn MCs.

Table 1 summarises all the learning algorithms available in Jajapy and AALpy for stochastic models. We can notice that Jajapy can learn non-deterministic models and CTMCs, which AALpy cannot. On the other hand, AALpy can learn non-stochastic models, which Jajapy cannot. Finally, in contrast to AALpy, Jajapy output models are immediately usable in Stormpy.

5.3.1 Jajapy Alergia vs AALpy Alergia

First, we compare Alergia implementations by Jajapy and AALpy for two Yao-Kunth'dice (one with $p = 0.5$ and one with $p = 0.9$), using two training sets containing each 10,000 traces of length 10. The results are summarised in Table 5 and 4.

There is no difference in the results obtained between these two implementations. Only the execution time differs: Jajapy being faster than AALpy.

Formula	Original	Jajapy	AALpy
# states	14	14	14
$\mathbb{P}[F(1)]$	0.167	0.164	0.164
$\mathbb{P}[F(2)]$	0.167	0.167	0.167
$\mathbb{P}[F(3)]$	0.167	0.165	0.165
$\mathbb{P}[F^{\leq 4}(3)]$	0.125	0.126	0.126
$\mathbb{P}[F^{\leq 4}(3)]$	0.125	0.125	0.125
$\mathbb{P}[F^{\leq 4}(3)]$	0.125	0.126	0.126
$\mathbb{P}[F^{\leq 10}(f)]$	0.996	0.996	0.996
ll distance	0.0	0.0	0.0
learning time	-	**0.001**	0.026

Table 4: Results for an unbiased Yao-Knuth's die ($p = 0.5$).

Formula	Original	Jajapy	AALpy
# states	14	14	14
$\mathbb{P}(F(1))$	0.801	0.807	0.807
$\mathbb{P}(F(2))$	0.089	0.088	0.088
$\mathbb{P}(F(3))$	0.01	0.01	0.01
$\mathbb{P}(F^{\leq 4}(3))$	0.081	0.076	0.076
$\mathbb{P}(F^{\leq 4}(3))$	0.009	0.007	0.007
$\mathbb{P}(F^{\leq 4}(3))$	0.001	0.001	0.001
$\mathbb{P}(F^{\leq 10}(f))$	1.0	1.0	1.0
ll distance	0.0	0.0	0.0
learning time	-	**0.001**	0.027

Table 5: Results for a biased Yao-Knuth's die ($p = 0.9$).

	true	AALpy	Jajapy BW
overall # of labels	-	74,285	74,285
# of traces	-	15,218	3,714
$\|S\|$ (# of states)	17	18	**17**
loglikelihood distance	0.0	0.7305	**0.3352**
$\mathbb{P}_{\max}[F^{\leq 4}(\text{goal})]$	0.336	0.322	**0.347**
$\mathbb{P}_{\max}[\neg \text{G } U^{\leq 4}(\text{goal})]$	0.072	0.074	0.074
Running time	-	**1.15 s**	290.8 s

Table 6: Results for the 3x3 deterministic grid-world model.

5.3.2 Jajapy BW vs AALpy L^*_{MDP}

We compare AALpy L^* and Jajapy BW algorithms on learning two variants of the grid-worlds presented in [12] and illustrated in Figure 7. A robot is moving in the grid, starting from the top-left corner. Its objective is to reach the bottom-right corner. The actions are the four directions —north, east, south, and west— and the observed labels are the different terrains. Depending on the target terrain, the robot may make errors in movement, e.g. move southwest instead of south. By construction, the 3x3 world is a deterministic MDP, and the 4x4 world is a non-deterministic one.

We run, for both models, AALpy for 200 L^* learning iterations and Jajapy for 200 BW iterations. We emphasize the fact that the two tools are using two different learning algorithms that are, in the author's opinion, complementary.

Table 6 and 7 show the results respectively for the 3x3 grid and the 4x4 grid. In both cases, the loglikelihood distance is computed for a test set containing 10,000 traces of length 20. First, we observe that, when the SUL is deterministic, the two output models are similar. Actually, Jajapy output is slightly closer to the SUL, but AALpy ran faster. However, when the SUL is non-deterministic, the difference between the two output models is more important. AALpy ran faster but produced a model with almost 8 times more states. Indeed L^*_{MDP}, by property, learned a deterministic approximation of the SUL, that is much bigger than the SUL itself. In terms of loglikelihood distance, AALpy output model

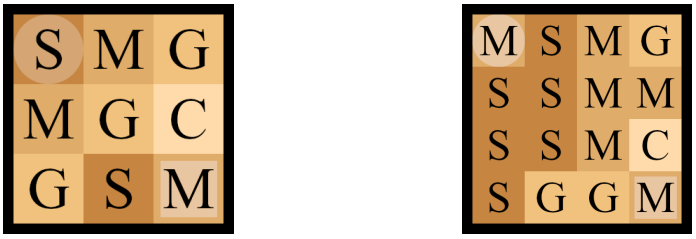

Figure 7: Grid worlds models. (Left) a 3x3 deterministic model; (Right) a 4x4 non-deterministic model.

	true	AALpy	Jajapy BW
overall # of labels	-	16,232,244	**200,000**
# of traces	-	$2,174,167$	**10,000**
$\|S\|$ (# of states)	28	207	**28**
loglikelihood distance	0.0	0.4963	**0.4680**
$\mathbb{P}_{\max}[F^{\leq 7}(\text{goal})]$	0.687	0.680	**0.692**
$\mathbb{P}_{\max}[F^{\leq 12}(\text{goal})]$	0.996	0.995	**0.996**
$\mathbb{P}_{\max}[\neg(\text{C} \mid \text{W})\ U^{\leq 7}(\text{goal})]$	0.520	**0.514**	0.504
Running time	-	**290.65 s**	15,303.83 s

Table 7: Results for the 4x4 non-deterministic grid-world model.

is slightly less accurate than Jajapy one. Finally, we notice that Jajapy uses far less information than AALpy, and does not require any interaction with the SUL (using a passive learning approach), in contrast to AALpy.

The fact that AALpy runs faster than Jajapy can be explained by the complexity of the two algorithms involved here, namely L^*_{MDP} and the BW algorithm. The BW algorithm is known to be costly in terms of time and memory complexity. Khreich et al. [43] point out several cases where, due to its cost, the BW algorithm could not be applied. In the same paper, they present a variant of it, requiring fewer memory resources while achieving the same results. However, Bartolucci et al. [44] show that this variant suffers from numerical problems.

In general, when learning MDPs, if it is impossible to interact with the SUL, we recommend Jajapy BW. Otherwise, we recommend using AALpy L^*_{MDP}, especially when the SUL is known to be deterministic.

6 Conclusions and Future Work

We presented Jajapy, a Python learning library for Markov models, and discussed its key features, implementation, usage, and performance evaluation. Jajapy is designed to be interoperable with Prism and Storm, and offers a variety of learning methods, both active and passive. We compared Jajapy and AALpy and argued that the two libraries complement each other, thus the choice of which library to use depends on the learning scenario.

As a future work, we consider implementing GPU-accelerated methods to speed-up the forward-backward computations required at each iteration of the BW algorithms borrowing ideas from [45, 46].

Data availability. An artifact allowing one to reproduce the experiments from this paper has been submitted to the QEST 2023 artifact evaluation.

References

[1] M. Kwiatkowska, G. Norman, and D. Parker, "Prism 4.0: Verification of probabilistic real-time systems," vol. 6806, 07 2011, pp. 585–591.

[2] C. Hensel, S. Junges, J.-P. Katoen, T. Quatmann, and M. Volk, "The probabilistic model checker storm," *International Journal on Software Tools for Technology Transfer*, vol. 24, pp. 1–22, 08 2022.

[3] R. Alur and T. A. Henzinger, "Reactive modules," *Formal Methods Syst. Des.*, vol. 15, no. 1, pp. 7–48, 1999. [Online]. Available: https://doi.org/10.1023/A:1008739929481

[4] C. E. Budde, A. Hartmanns, M. Klauck, J. Křetínský, D. Parker, T. Quatmann, A. Turrini, and Z. Zhang, "On correctness, precision, and performance in quantitative verification - qcomp 2020 competition report," in *Leveraging Applications of Formal Methods*, 2020.

[5] R. Reynouard. (2022) Jajapy github repository. [Online]. Available: https://github.com/Rapfff/jajapy

[6] R. C. Carrasco and J. Oncina, "Learning Stochastic Regular Grammars by Means of a State Merging Method," in *ICGI-94*, ser. LNCS, R. C. Carrasco and J. Oncina, Eds., vol. 862. Springer, 1994, pp. 139–152.

[7] ——, "Learning deterministic regular grammars from stochastic samples in polynomial time," *RAIRO – Theoretical Informatics and Applications (RAIRO: ITA)*, vol. 33, no. 1, pp. 1–20, 1999.

[8] H. Mao, Y. Chen, M. Jaeger, T. Nielsen, K. Larsen, and B. Nielsen, "Learning probabilistic automata for model checking," 10 2011, pp. 111 – 120.

[9] H. Mao, Y. Chen, M. Jaeger, T. D. Nielsen, K. G. Larsen, and B. Nielsen, "Learning Deterministic Probabilistic Automata from a Model Checking Perspective," *Machine Learning*, vol. 105, no. 2, pp. 255–299, 2016.

[10] L. R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, Feb 1989.

[11] G. Bacci, A. Ingólfsdóttir, K. G. Larsen, and R. Reynouard, "Active learning of markov decision processes using baum-welch algorithm (extended)," *CoRR*, vol. abs/2110.03014, 2021. [Online]. Available: https://arxiv.org/abs/2110.03014

[12] G. Bacci, A. Ingólfsdóttir, K. G. Larsen, and R. Reynouard, "Active learning of markov decision processes using baum-welch algorithm," in *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021, pp. 1203–1208.

[13] ——, "Mm algorithms to estimate parameters in continuous-time markov chains," 2023. [Online]. Available: https://arxiv.org/abs/2302.08588

[14] R. Reynouard. (2022) Jajapy's documentation. [Online]. Available: https://jajapy.readthedocs.io/en/latest/

[15] ——, "A short introduction to jajapy," Mar. 2023. [Online]. Available: https://doi.org/10.5281/zenodo.7695105

[16] E. Muškardin, B. Aichernig, I. Pill, A. Pferscher, and M. Tappler, "Aalpy: an active automata learning library," *Innovations in Systems and Software Engineering*, vol. 18, pp. 1–10, 03 2022.

[17] M. Tappler, E. Muskardin, B. K. Aichernig, and I. Pill, "Active model learning of stochastic reactive systems," in *Software Engineering and Formal Methods - 19th International Conference, SEFM 2021, Virtual Event, December 6-10, 2021, Proceedings*, ser. Lecture Notes in Computer Science, R. Calinescu and C. S. Pasareanu, Eds., vol. 13085. Springer, 2021, pp. 481–500. [Online]. Available: https://doi.org/10.1007/978-3-030-92124-8_27

[18] D. Angluin, "Learning Regular Sets from Queries and Counterexamples," *Information and Computation*, vol. 75, no. 2, pp. 87–106, 1987.

[19] M. Isberner, F. Howar, and B. Steffen, "The open-source learnlib," in *Computer Aided Verification*, D. Kroening and C. S. Păsăreanu, Eds. Cham: Springer International Publishing, 2015, pp. 487–495.

[20] B. Bollig, J.-P. Katoen, C. Kern, M. Leucker, D. Neider, and D. Piegdon, "libalf: The automata learning framework," 07 2010, pp. 360–364.

[21] F. Thollard, P. Dupont, and C. de la Higuera, "Probabilistic dfa inference using kullback-leibler divergence and minimality," in *International Conference on Machine Learning*, 2000.

[22] A. Stolcke, "Bayesian learning of probabilistic language models," 1994.

[23] A. Gelfand and A. Smith, "Sampling-based approaches to calculate marginal densities," *Journal of the American Statistical Association*, vol. 85, pp. 398–409, 06 1990.

[24] R. Neal, "Markov chain sampling methods for dirichlet process mixture models," *Journal of Computational and Graphical Statistics*, vol. 9, pp. 249–265, 06 2000.

[25] P. Rashidinejad, B. Zhu, C. Ma, J. Jiao, and S. J. Russell, "Bridging offline reinforcement learning and imitation learning: A tale of pessimism," *IEEE Transactions on Information Theory*, vol. 68, pp. 8156–8196, 2021.

[26] Y. Jin, Z. Yang, and Z. Wang, "Is pessimism provably efficient for offline rl?" in *International Conference on Machine Learning*, 2020.

[27] J. Buckman, C. Gelada, and M. G. Bellemare, "The importance of pessimism in fixed-dataset policy optimization," *ArXiv*, vol. abs/2009.06799, 2020.

[28] J. Morimoto and K. Doya, "Robust Reinforcement Learning," *Neural Computation*, vol. 17, no. 2, pp. 335–359, 02 2005. [Online]. Available: https://doi.org/10.1162/0899766053011528

[29] S. H. Lim, H. Xu, and S. Mannor, "Reinforcement learning in robust markov decision processes," in *Advances in Neural Information Processing Systems*, C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, Eds., vol. 26. Curran Associates, Inc., 2013. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2013/file/0deb1c54814305ca9ad266f53bc82511-Paper.pdf

[30] M. Suilen, T. D. Simão, D. Parker, and N. Jansen, "Robust anytime learning of markov decision processes," 2023.

[31] E. Derman, D. J. Mankowitz, T. A. Mann, and S. Mannor, "A bayesian approach to robust reinforcement learning," in *Conference on Uncertainty in Artificial Intelligence*, 2019.

[32] M. Češka, C. Hensel, S. Junges, and J.-P. Katoen, "Counterexample-guided inductive synthesis for probabilistic systems," *Form. Asp. Comput.*, vol. 33, no. 4–5, p. 637–667, aug 2021. [Online]. Available: https://doi.org/10.1007/s00165-021-00547-2

[33] T. Quatmann, C. Dehnert, N. Jansen, S. Junges, and J. Katoen, "Parameter synthesis for markov models: Faster than ever," *CoRR*, vol. abs/1602.05113, 2016. [Online]. Available: http://arxiv.org/abs/1602.05113

[34] N. Jansen, S. Junges, and J. Katoen, "Parameter synthesis in markov models: A gentle survey," in *Principles of Systems Design - Essays Dedicated to Thomas A. Henzinger on the Occasion of His 60th Birthday*, ser. Lecture Notes in Computer Science, J. Raskin, K. Chatterjee, L. Doyen, and R. Majumdar, Eds., vol. 13660. Springer, 2022, pp. 407–437. [Online]. Available: https://doi.org/10.1007/978-3-031-22337-2_20

[35] E. Polgreen, V. B. Wijesuriya, S. Haesaert, and A. Abate, "Data-efficient bayesian verification of parametric markov chains," in *Quantitative Evaluation of Systems - 13th International Conference, QEST 2016, Quebec City, QC, Canada, August 23-25, 2016, Proceedings*, ser. Lecture Notes in Computer Science, G. Agha and B. V. Houdt, Eds., vol. 9826. Springer, 2016, pp. 35–51. [Online]. Available: https://doi.org/10.1007/978-3-319-43425-4_3

[36] L. Baum, T. Petrie, G. Soules, and N. Weiss, "A maximization technique occurring in the statistical analysis of probabilistic functions of markov chains," 1970.

[37] F. Yang, S. Balakrishnan, and M. J. Wainwright, "Statistical and computational guarantees for the baum-welch algorithm," *Journal of Machine Learning Research*, vol. 18, no. 125, pp. 1–53, 2017. [Online]. Available: http://jmlr.org/papers/v18/16-093.html

[38] W. Hoeffding, "Probability inequalities for sum of bounded random variables," 1961.

[39] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[40] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[41] D. Knuth and A. Yao, *Algorithms and Complexity: New Directions and Recent Results*. Academic Press, 1976, ch. The complexity of nonuniform random number generation.

[42] H. Hermanns, J. Meyer-Kayser, and M. Siegle, "Multi terminal binary decision diagrams to represent and analyse continuous time Markov chains," in *Proc. 3rd International Workshop on Numerical Solution of Markov Chains (NSMC'99)*, B. Plateau, W. Stewart, and M. Silva, Eds. Prensas Universitarias de Zaragoza, 1999, pp. 188–207.

[43] W. Khreich, E. Granger, A. Miri, and R. Sabourin, "On the memory complexity of the forward–backward algorithm," *Pattern Recognition Letters*, vol. 31, no. 2, pp. 91–99, 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167865509002578

[44] F. Bartolucci and S. Pandolfi, "Comment on the paper "on the memory complexity of the forward–backward algorithm," by khreich w., granger e., miri a., sabourin, r." *Pattern Recognition Letters*, vol. 38, pp. 15–19, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167865513003863

[45] Y. Shao, Y. Wang, D. Povey, and S. Khudanpur, "Pychain: A fully parallelized pytorch implementation of LF-MMI for end-to-end ASR," in *Interspeech 2020, 21st Annual Conference of the International Speech Communication Association, Virtual Event, Shanghai, China, 25-29*

October 2020, H. Meng, B. Xu, and T. F. Zheng, Eds. ISCA, 2020, pp. 561–565. [Online]. Available: https://doi.org/10.21437/Interspeech.2020-3053

[46] L. Ondel, L. Lam-Yee-Mui, M. Kocour, C. F. Corro, and L. Burget, "Gpu-accelerated forward-backward algorithm with application to lattice-free MMI," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022, Virtual and Singapore, 23-27 May 2022*. IEEE, 2022, pp. 8417–8421. [Online]. Available: https://doi.org/10.1109/ICASSP43922.2022.9746824